

Project: Ambient SMS Agent (Multi-Tenant AI Swarm)

1. Executive Summary

The Concept: An AI agent that lives as a standard phone contact. Users can add this agent to native iOS/Android MMS group chats. It operates ambiently, only waking up when @mentioned.

The Magic: It is multi-tenant and deeply contextual. When Alice @mentions the agent in a group chat, it uses Alice's semantic memory and authorized tools (like her GitHub or Calendar). When Bob @mentions it in the same chat, it switches to Bob's context. **The Theme:** "Leaving the chatbox." The agent meets the user where work and life already happen (native text messaging), requiring zero app downloads while maintaining enterprise-grade security and personalized memory.

2. The Core Tech Stack

- **Transport & Auth:** Twilio (SMS/MMS Webhooks, Verify API for OTP).
- **Backend Framework:** Python / FastAPI (handles webhooks, routing, and Web UI).
- **AI Orchestration:** PydanticAI (handles structured outputs, LLM routing, and dynamic sub-agent instantiation).
- **Memory / State:** Cortex (Nuclear Notes) acting as an external semantic memory graph, accessed via the Model Context Protocol (MCP).
- **External Tooling:** Standard MCP servers (e.g., GitHub, Web Search) dynamically mounted based on user permissions.

3. The User Experience (UX) Flow

1. **The Trigger:** A user adds the Agent's phone number to a group chat and texts: "*@agent can you summarize the latest PR on my backend repo?*"
2. **JIT Onboarding (If New User):** If the agent doesn't recognize the sender's phone number, it replies: "*I don't have a workspace for you yet! Tap here to authenticate: [portal.link]*"
3. **The Identity Bridge:** The user taps the link, enters their phone number, and confirms a 6-digit SMS OTP. They are now securely logged in.
4. **Tool Connection:** In the Web UI, the user clicks "Connect GitHub." (For the hackathon, this is a simulated OAuth button that drops a mock token into their profile).
5. **The Execution:** The user returns to the group chat. The agent processes the request, seamlessly utilizing their personal memory (Cortex) to know *which* repo is the "backend repo," and their GitHub tool to read the PR. It drops the summary back into the group chat.

4. System Architecture (Zero-Trust)

This project utilizes a **Zero-Trust, Event-Sourced Swarm Architecture**. To prevent context-window bloat and secure personal data, the main AI Orchestrator has *no tool access*. It delegates entirely to isolated sub-agents.

Component Breakdown:

- **The Gateway (FastAPI):** Receives the Twilio webhook. Aggressively filters out messages without @agent (returning 200 OK instantly to prevent token burn). Uses the sender's phone number (From) as the universal Primary Key for all downstream actions.
- **The Orchestrator (PydanticAI):** The "manager." It receives the prompt but has zero tools. It analyzes the request and outputs a structured JSON plan to spawn specialized Sub-Agents.
- **Sub-Agents (Ephemeral):** Short-lived LLM instances given specific, isolated access.
 - *Context Agent:* Only mounted with the **Cortex MCP** tool. Used to look up personal user preferences and past context.
 - *Execution Agent:* Mounted with external MCP tools (e.g., **GitHub MCP**). Operates using the specific OAuth token belonging to the phone number that initiated the request.
- **The Memory Extractor (Async Sub-routine):** A separate background agent that runs *after* the user receives their text reply.

5. End-to-End Implementation Workflows

Workflow A: Message Routing & Multi-Tenancy

1. Twilio receives an MMS group text and sends an HTTP POST to the FastAPI /webhook/sms endpoint.
2. FastAPI parses the Body (message), From (sender), and To (group thread).
3. If @agent is missing, drop the request.
4. If @agent is present, look up the From phone number in the local database/state to fetch the user's active tool tokens (e.g., github_token).
5. If the user doesn't exist, trigger Twilio Verify API to send an OTP and text back an onboarding link. Pause execution.

Workflow B: The Zero-Trust Execution Loop

1. FastAPI passes the user's prompt and available tool list to the Orchestrator.
2. The Orchestrator LLM decides what needs to happen. (e.g., *"I need to check Cortex to see what repo the user cares about, then check GitHub."*)
3. The Orchestrator outputs a structured plan to FastAPI.
4. FastAPI dynamically spins up the **Context Sub-agent**, injecting the Cortex MCP tool. It retrieves semantic facts about the user.
5. FastAPI dynamically spins up the **Execution Sub-agent**, injecting the GitHub MCP tool and the user's specific GitHub token. It fetches the repo data.
6. FastAPI returns both sub-agent results to the Orchestrator for a final synthesis.
7. FastAPI sends the synthesized text back to Twilio as an XML <Response>.

Workflow C: Asynchronous Memory Extraction

We do not want the Orchestrator manually managing memory during execution—it causes latency and hallucination risks.

1. Once the text is sent back to the user, a background task fires in FastAPI.
2. The **Memory Extractor Sub-agent** (a fast model like gpt-4o-mini) is fed the entire transcript of the recent interaction.
3. It uses a strict structured output schema (Pydantic model) to extract `novel_facts`, `preferences`, and a `should_update` boolean.
4. If `should_update` is true, the backend calls the Cortex API to permanently store these new facts in the user's semantic workspace, grounding the AI for future interactions.

6. Hackathon Demo Strategy

To maximize the "wow factor" for the judges:

1. **Live Dashboard:** Keep a terminal or web dashboard on the projector showing live webhook events, Sub-agent dispatches, and Memory extractions in real-time.
2. **The "Room Test":** Show a QR code/phone number on screen at the end of the presentation. Encourage the entire audience to text the agent right then.
3. **Organic JIT Demo:** Because none of the audience members are in the system, they will all instantly experience the JIT onboarding flow, proving the system is live, multi-tenant, and dynamically scalable.